National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	06
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	Introduction to MySQL & Application Development
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

1 Introduction to MySQL 1
1.1 Why Move from SQLite to MySQL? 1
1.2 What is MySQL? 1
1.3 SQLite vs MySQL 2
1.4 MySQL Architecture Overview 2
1.5 MySQL Data Types 2
1.5.1 Numeric 2
1.5.2 String 3
1.5.3 Date & Time 3
1.6 Auto-incremented Columns 4
1.7 TRUNCATE TABLE 4
1.8 Named Constraints 4
1.9 ALTER TABLE 5
1.10 Getting Started with MySQL 7
1.10.1 Installation & Setup 7
1.10.2 Working with MySQL 7
1.10.2.1 Create a Database 8
1.10.2.2 View Databases 8
1.10.2.3 Select Database 8
1.10.2.4 Creating a Table 8
1.10.2.5 Delete a Database 9
2 Application Development 9
2.1 What is PHP? 9
2.2 Running PHP 10
2.2.1 Using XAMPP 10
2.2.2 PHP CLI 10
2.3 Basic PHP Syntax 10
2.4 Variables & Constants 11
2.4.1 Variables 11
2.4.2 Constants 11
2.5 Input/Output Functions 11
2.6 Data Types 12
2.7 Operators 13
2.8 Comments 14
2.9 Arrays 15
2.9.1 Indexed Array 15
2.9.2 Associative Array 15
2.10 Conditional Statements 16
2.11 Iteration Statements 17
2.12 Functions 18
2.13 Exception Handling 18

1 Introduction to MySQL

Up to this point, we have been working with **SQLite**, a lightweight, file-based relational database system. Now, we transition to **MySQL**, a powerful, server-based RDBMS used in real-world web applications and enterprise systems.

1.1 Why Move from SQLite to MySQL?

SQLite is excellent for:

- Learning SQL fundamentals
- Small applications
- Embedded systems
- Local development
- Single-user environments

However, MySQL is preferred when:

- Multiple users access the database simultaneously
- You need high performance and scalability
- You need better security and user management
- You need advanced features (stored procedures, triggers, indexing options, etc.)
- You are building web applications (PHP + MySQL is extremely common)

1.2 What is MySQL?

MySQL is an open-source, server-based Relational Database Management System (RDBMS).

Unlike SQLite:

- It runs as a **database server**
- Clients connect using username & password
- Databases are not just files, instead they are managed by the server

It is widely used in:

- Web development (PHP, Laravel, Node.js, etc.)
- Enterprise applications

- Cloud infrastructure
- LAMP stack (Linux, Apache, MySQL, PHP)

1.3 SQLite vs MySQL

DBMS	Architecture	Installation	Concurrency	User Management	Data Types
SQLite	File-based	Zero configuration	Limited	None	Dynamic typing
MySQL	Client-server	Server installation	High concurrency	Full user & rights system	Strict typing

1.4 MySQL Architecture Overview

MySQL follows a **Client-Server** architecture. It is composed of the following components:

1. **MySQL Server**
   - Manages databases
   - Handles queries
   - Controls users and permissions

2. **Client Tools**
   - MySQL CLI
   - MySQL Workbench
   - phpMyAdmin
   - Application programs

1.5 MySQL Data Types

Unlike SQLite (dynamic typing), MySQL uses **strict and well-defined data types**.

1.5.1 Numeric

Some of the standard numeric data types are listed below:

Data Type	Description / Use

`INT`	Standard integer
`DECIMAL(total_digits, digits_after_decimal)`	Exact fixed-point numbers (e.g., money values)
`DOUBLE`	Large precision floating number
`BOOLEAN`	True/False (internally `TINYINT`)

1.5.2 String

Some of the standard string data types are listed below:

Storage Class	**Description / Use**
`CHAR(n)`	Fixed-length string of maximum `n` characters
`VARCHAR(n)`	Variable-length string of maximum `n` characters
`TEXT`	Large text content
`ENUM(val1, val2, ...)`	Restricts column to predefined values

1.5.3 Date & Time

SQLite stores dates as a string or an integer data. In contrast, MySQL has true date types.

Storage Class	**Description / Use**
`DATE`	Stores date (YYYY-MM-DD)
`TIME`	Stores time (HH:MM:SS)
`DATETIME`	Date + Time
`TIMESTAMP`	DateTime with auto-update capability
`YEAR`	Stores year

1.6 Auto-incremented Columns

Unlike SQLite, MySQL allows any one integer column per table (generally the primary key) to be set as auto-incremented through the following syntax:

Examples:

```
CREATE TABLE table_name (
    id INT PRIMARY KEY AUTO_INCREMENT -- For primary key column
);

-- OR

CREATE TABLE table_name (
    id CHAR(4) PRIMARY KEY,
    serial INT AUTO_INCREMENT -- For non-primary key column
);
```

1.7 TRUNCATE TABLE

`TRUNCATE TABLE` removes **all rows** from a table quickly and efficiently. It resets the table to its initial empty state.

It is different from `DELETE` in that it cannot be used with a `WHERE` clause. Also, `TRUNCATE` is generally faster and potentially more dangerous.

General Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE students;
```

1.8 Named Constraints

Unlike SQLite (which mostly ignores names), MySQL fully supports **named constraints**.

Named constraints are constraints on columns defined at the table level during table creation. Following are the advantages of using named constraints instead of inline constraints:

- Easier to identify errors
- Easier to modify or drop later
- More readable schema
- Required in large systems

General Syntax:

```
CREATE TABLE table_name (
    column_name DATATYPE,
    CONSTRAINT constraint_name CONSTRAINT_TYPE
);
```

Example:

```
CREATE TABLE students (
    id INT,
    email VARCHAR(100),
    CONSTRAINT pk_students PRIMARY KEY (id),
    CONSTRAINT uq_students_email UNIQUE (email)
);
```

1.9 ALTER TABLE

SQLite has very limited ALTER support. MySQL supports many modifications.

General Syntax:

```
-- Multiple actions can be combined in a single statement
ALTER TABLE table_name
    action1,
    action2,
    ...;
```

Examples:

-

```
-- Add a column at specified column position
ALTER TABLE table_name
ADD column_name data_type [constraints]
    [FIRST | AFTER existing_column];

-- Drop a column
ALTER TABLE table_name
DROP COLUMN column_name;
```

-

```
-- Modify a column (change datatype / constraints)
ALTER TABLE table_name
MODIFY column_name new_data_type [constraints];
```

-

```
-- Change a column (rename + redefine)
ALTER TABLE table_name
CHANGE old_column_name new_column_name data_type [constraints];
```

-

```
-- Add constraint on a column
ALTER TABLE table_name
ADD CONSTRAINT constraint_name
CONSTRAINT_TYPE (column_name);

-- Drop a column's constraint
ALTER TABLE table_name
DROP INDEX constraint_name;
```

-

```
-- Rename s table
ALTER TABLE old_table_name
RENAME TO new_table_name;
```

1.10 Getting Started with MySQL

1.10.1 Installation & Setup

There are two common ways to install MySQL:

- **Option 1: Install MySQL from Official Website**
  a. Download MySQL Community Server
  b. Install MySQL Server
  c. Install MySQL Workbench (GUI tool)

- **Option 2: Install via Software Suites (e.g XAMPP, WAMP, MAMP etc.)**
  a. Download XAMPP
  b. Install it

What is XAMPP?

XAMPP is a free software package that allows you to run a **local web server environment** on your computer for development and testing.

It includes:

- **X** (Cross-platform)
- **A** (Apache)
- **M** (MySQL / MariaDB in newer versions)
- **P** (PHP & phpMyAdmin)
- **P** (Perl)

1.10.2 Working with MySQL

If using XAMPP:

- Open XAMPP Control Panel
- Click the Start button next to MySQL
- Wait until the service turns green (Running)
- Click the Shell button to launch the XAMPP command-line.

The typical work-flow of working with MySQL is:

- Inside the XAMPP shell, access the MySQL CLI by running the following:

```
mysql -u root -p
```

- Press Enter (default password may be empty).
- You are now connected to MySQL server as root.

1.10.2.1 Create a Database

```
CREATE DATABASE database_name;
```

1.10.2.2 View Databases

```
SHOW DATABASES;
```

1.10.2.3 Select Database

Unlike SQLite (which opens a file), MySQL requires selecting a database before creating tables in it.

```
USE database_name;
```

1.10.2.4 Creating a Table

After selecting a database, we can work exactly like SQLite:

Example:

```
CREATE TABLE employees (
    emp_id INT AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    salary DECIMAL(10,2) NOT NULL,
    department ENUM('CS','SE'),
    status ENUM('Active','Inactive') DEFAULT 'Active',
    hire_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_employees PRIMARY KEY (emp_id),
    CONSTRAINT uq_employees_email UNIQUE (full_name),
    CONSTRAINT chk_salary CHECK (salary > 0)
);
```

Additionally, we can use the the following commands to view the structure of a table:

```
DESCRIBE employees;
-- OR
DESC employees; -- (Short-form)
```

1.10.2.5 Delete a Database

If needed, MySQL also provides the capability to completely remove a database along with all its data.

In order to delete an entire database, run:

```
DROP database_name;  -- Extremely dangerous
```

2 Application Development

Application Development refers to the process of designing, creating, testing, and maintaining software applications to solve specific problems or perform particular tasks.

In the current and upcoming labs, Application Development focuses on using **PHP** as the server-side programming language together with **MySQL** as the database management system to build dynamic web applications.

2.1 What is PHP?

PHP **(PHP Hypertext Preprocessor)** is a server-side scripting language primarily used to build dynamic web applications.

Unlike C/C++ or Python (which typically run as standalone programs), PHP:

- Executes on a **web server**
- Generates **HTML output**
- Is embedded inside HTML
- Is widely used with MySQL

When a browser requests a `.php` file:

1. The web server (e.g., Apache via XAMPP) receives the request.
2. PHP engine executes the PHP code.

3. The server sends **generated HTML** back to the browser.
4. The browser never sees PHP code but only the output

2.2 Running PHP

2.2.1 Using XAMPP

- Open XAMPP Control Panel
- Click the Start button next to Apache
- Place PHP files inside: *C:\xampp\htdocs*
- Open a web browser and enter the URL: *http://localhost/filename.php*
- The output of the PHP script will be displayed on the webpage in the form of HTML.

2.2.2 PHP CLI

- PHP can also run in the terminal if added to PATH.
- Open terminal and run your script with the command: *php filename.php*
- This is useful for learning fundamentals without web context.

2.3 Basic PHP Syntax

- PHP filenames end with an extension of `.php`. PHP code is written inside:

```
<?php
    // PHP code
?>
```

- Every PHP statement must end with a semicolon.
- A PHP script can contain:
    - Pure PHP
    - Pure HTML
    - Or both mixed

Example:

```
<h1>Welcome</h1>

<?php
```

```
echo "Hello World! <br>";
?>
```

2.4 Variables & Constants

2.4.1 Variables

- All variable names start with the dollar/money sign $.
- No type declaration is required
- Variable names are case-sensitive

Example:

```
<?php
$name = 'Ali';
$age = 20;
?>
```

2.4.2 Constants

- Its value cannot be changed.
- Created using the `defined` function.
- Can also be defined using the `const` keyword.
- The convention is to have the constant name in uppercase.

Example:

```
<?php
define('PI', 3.14);
const SITE_NAME = 'My Website';

echo PI, SITE_NAME;
?>
```

2.5 Input/Output Functions

Construct / Function	Description
`readline()`	Reads input from CLI
`echo`	Outputs multiple values passed as comma-separated arguments
`print`	Outputs only one value (returns 1)
`var_dump()`	Detailed variable info (type + value)
`print_r()`	Human-readable array/object output

Example:

```
<?php
$language = 'PHP';
$name = readline('Enter your name: ');

print "Hi, ${name}\n";
echo 'Do you still hate ' . $language . "?\n";

var_dump($name);
print_r([1,2,3]);
?>
```

2.6 Data Types

PHP has a dynamic typing system. However, the following data types are supported.

Type	Description
`int`	Whole numbers
`float / double`	Decimal numbers
`string`	Text values
`boolean`	True or false
`array`	Multiple values (collection)
`null`	No value (empty variable)

`object`	Stores data as objects
`resource`	References external resources

Example:

```
<?php
$age = 12;
$gpa = 3.4;
$name = 'Ahmad';
$has_warning = false;
$marks = [74, 85, 71];
$empty = null;

echo gettype($marks), ' ', gettype($empty);
?>
```

2.7 Operators

Type	Description	Example
Arithmetic Operators		
`+`	Addition	`echo 5 + 1;`
`-`	Subtraction	`echo 11 - 8;`
`*`	Multiplication	`echo 4 * 3;`
`/`	Division	`echo 10 / 3;`
`%`	Modulus	`echo 6 % 5;`
`**`	Power	`echo 2 ** 6;`
Assignment Operators		
`=`	Assign	`$var1 = 1; $var2 = 'Tw';`
`+=`	Add and assign	`$var1 += 4;`
`-=`	Subtract and assign	`$var1 -= 7;`

`.=`	Concatenate string and assign	`$var2 .= 'o';`
Comparison Operators		
`==`	Equal (value only)	`echo 1 == '1';`
`===`	Strict equal (value + type)	`echo '1' === '1';`
`!=`	Not equal	`echo 1 != '1';`
`!==`	Strict not equal	`echo '1' !== '1';`
`>, <, >=, <=`	Standard comparisons	`echo 4 >= 3;`
Logical Operators		
`and / &&`	Logical AND	`echo 3 > 2 && 2 > 1;`
`or / ||`	Logical OR	`echo 3 > 2 || 1 > 2;`
`!`	Logical NOT	`echo !(3<2);`
Concatenation Operators		
`.`	String concatenation	`echo 'One' . 'Two';`
`" ${variable}"`	String interpolation	`echo "One ${var2}";`
Ternary Operator		
`? :s`	Short-hand if-else statement.	`$result = ($age >= 18) ? "Adult" : "Minor";`
Null Coalescing Operator		
`??`	Returns the RHS value if the LHS value is null or not set.	`$username = $input ?? "Guest";`

2.8 Comments

Type	Syntax
Single line	`// comment`
Single line	`# comment`

Multi-line	`/* comment */`

2.9 Arrays

2.9.1 Indexed Array

Arrays are zero-indexed in PHP.

Example:

```
<?php
// Creating arrays
$numbers = [10, 20, 30];
$fruits = array('Apple', 'Banana');

// Accessing elements
echo $numbers[0];
echo $fruits[1];
?>
```

2.9.2 Associative Array

Associative arrays consist of key-value pairs. In other words, the array elements are indexed by user-defined keys (strings).

Example:

```
<?php
// Creating associative arrays
$student = [
    'name' => 'Ali',
    'age' => 20,
    'cgpa' => 3.5
];
$credentials = array(
    'username' => 'ali',
    'password' => '123'
);
```

```
// Accessing associative elements
echo $student['name'], $student['age'], $student['cgpa'];
?>
```

2.10 Conditional Statements

```
<?php
$age = 17;
$grade = 'B';

// If Statement
if ($age > 18) {
    echo 'Adult';
}

// If-Else Statement
if ($age > 18) {
    echo 'Adult';
} else {
    echo 'Not an Adult';
}

// If-Elseif-Else Statement
if ($age > 18) {
    echo 'Adult';
} elseif ($age > 12 {
    echo 'Teenager';
} else {
    echo 'Child';
}

// Switch Statement
switch ($grade) {
    case 'A':
        echo 'Excellent';
        break;
    default:
```

```
        echo 'Unknown';
}
?>
```

2.11 Iteration Statements

```
<?php
// For-each loop
# Use with indexed array
foreach ($numbers as $value) {
    echo $value;
}
# Use with associative array
foreach ($student as $key => $value) {
    echo "${key}: ${value}\n";
}

// While loop
$i = 0;
while ($i < 5) {
    echo $i;
    $i++;
}

// Do-while loop
do {
    echo $i;
    $i++;
} while ($i < 10)

// For loop
for ($i = 0; $i < 5; $i++) {
    echo $i;
}
?>
```

2.12 Functions

```php
<?php
// Define function
function clean($input) {
    return trim($input);
}

// Default parameters
function welcome($name = 'Guest') {
    echo 'Welcome ' . $name;
}

// Call function
echo clean(' Ahmad ');
welcome();
?>
```

2.13 Exception Handling

PHP supports try-catch blocks to gracefully handle unexpected errors to avoid application crashes and easily find bugs.

```php
<?php
$age = 0;
try {
    // Include code segments expected to cause errors
    if ($age < 0) {
        // Custom user-defined exception for demonstration
        throw new Exception("Invalid age");
    }
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}
?>
```